CHAPTER 8

The Simply Typed Lambda Calculus and Closures

The untyped λ-calculus of Chapter 7 can compute anything, and it cannot tell anything apart. The boolean $false$ and the number $\overline{0}$ are the same term; $plus$ applied to a pair produces an answer; $\lambda x.\, x\, x$ applies a function to itself, and Ω runs forever. In 1940 Alonzo Church added **types** to his calculus, and most of this strangeness disappeared at once. The result, the **simply typed λ-calculus** (STLC), is the smallest interesting type theory, the core of every typed functional language, and the system that Chapter 9 proves safe and Chapter 10 reads as logic.

Rust's functions are typed λ-terms too, but a Rust programmer meets function types in several costumes that the clean theory does not have: `fn` items, function pointers, closures with anonymous types, `impl Fn`, `dyn Fn`, and three traits, `Fn`, `FnMut` and `FnOnce`. The second half of this chapter explains them one by one, and shows that the three traits answer a question the λ-calculus never asks: how many times may a function be called? That question leads straight to Part V.

In this chapter you will learn:

- the types and typing rules of the STLC with unit, booleans, naturals, pairs and sums, as introduction and elimination rules;
- the computation rules β and η, why every well-typed term terminates, and why $\lambda x.\, x\, x$ has no type;
- bidirectional type checking, where λ checks but does not synthesize, and how the book's `stlc` crate implements it with an evaluator based on closures;
- what Rust's `fn` items, function pointers, closures, `impl Fn` and `dyn Fn` are, and what each costs;
- capture modes, `move`, and the `Fn` / `FnMut` / `FnOnce` hierarchy read as "how many times may this resource be used";
- why currying, free in the λ-calculus, has a price in Rust;
- how Lean represents intrinsically typed terms whose evaluator is total and safe by construction, and how that differs from the extrinsic style of Chapter 9.

8.1 Types for the λ-calculus

The types are built from three base types and three type formers:

$$\tau, \sigma ::= \texttt{Unit} \mid \texttt{Bool} \mid \texttt{Nat} \mid \sigma \to \tau \mid \sigma \times \tau \mid \sigma + \tau$$

$\sigma \to \tau$ is the type of functions from σ to τ, $\sigma \times \tau$ the type of pairs, and $\sigma + \tau$ the type of **tagged unions**: a value of $\sigma + \tau$ is either a σ tagged "left" or a τ tagged "right", like a Rust `enum Either<S, T> { Left(S), Right(T) }`. The arrow associates to the right, $\texttt{Nat} \to \texttt{Nat} \to \texttt{Nat}$ meaning $\texttt{Nat} \to (\texttt{Nat} \to \texttt{Nat})$, the type of curried two-argument functions.

The terms are those of Chapter 7, with an annotation on the parameter of each λ, plus constants and operations for the new types:

$$t ::= x \mid \lambda x : \tau . t \mid t\ t \mid () \mid \texttt{true} \mid \texttt{false} \mid \texttt{if}\ t\ \texttt{then}\ t\ \texttt{else}\ t$$

$$\mid 0 \mid \texttt{succ}\ t \mid \texttt{iter}\ t\ t\ t \mid (t, t) \mid \texttt{fst}\ t \mid \texttt{snd}\ t$$

$$\mid \texttt{inl}\ t \mid \texttt{inr}\ t \mid \texttt{case}\ t\ \texttt{of}\ \texttt{inl}\ x \Rightarrow t \mid \texttt{inr}\ y \Rightarrow t$$

Most of these are familiar. `iter n z s` applies the function `s` to `z`, `n` times: it is the way to compute with a natural number, the typed version of a Church numeral (Chapter 7), and the same thing as `(0..n).fold(z, |acc, _| s(acc))` in Rust. `case` is a two-armed `match` on a sum, binding `x` in the first arm and `y` in the second.

Annotating every parameter, as in $\lambda x : \texttt{Nat}.\ \texttt{succ}\ x$, is called the **Church style** of typing: types are part of the terms. In the **Curry style** terms stay unannotated and a type is something you assign to them afterwards, possibly in several ways ($\lambda x. x$ has every type $\tau \to \tau$). We will use both: Church style for the rules, Curry style in the Lean theorem about self-application, and a mixture, bidirectional checking, in the Rust checker.

8.2 The typing rules

The judgment is $\Gamma \vdash t : \tau$, as in Chapter 3, with a context $\Gamma = x_1 : \tau_1, \ldots, x_n : \tau_n$ that later entries may shadow. The heart of the system is three rules, for variables, abstraction and application:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \text{T-Var} \qquad \frac{\Gamma, x : \sigma \vdash t : \tau}{\Gamma \vdash \lambda x : \sigma . t : \sigma \to \tau}\ \text{T-Lam} \qquad \frac{\Gamma \vdash t : \sigma \to \tau \qquad \Gamma \vdash u : \sigma}{\Gamma \vdash t\ u : \tau}\ \text{T-App}$$

Rule T-Lam says: to type a function, assume its parameter has the annotated type and type the body. Rule T-App says: a function of type $\sigma \to \tau$ applied to a σ gives a τ. You have relied on both every time Rust checked a closure: `|x: u32| x + 1` has type "`u32` to `u32`" because, with `x: u32`, the body is a `u32`.

The other types follow one pattern. Each type former has **introduction** rules, which build values of the type, and **elimination** rules, which use them. For booleans, naturals, pairs and sums:

$$\frac{}{\Gamma \vdash \texttt{true} : \texttt{Bool}}\ \text{T-True} \qquad \frac{\Gamma \vdash c : \texttt{Bool} \qquad \Gamma \vdash t : \tau \qquad \Gamma \vdash u : \tau}{\Gamma \vdash \texttt{if}\ c\ \texttt{then}\ t\ \texttt{else}\ u : \tau}\ \text{T-If}$$

$$\frac{}{\Gamma \vdash 0 : \texttt{Nat}}\ \text{T-Zero} \qquad \frac{\Gamma \vdash n : \texttt{Nat}}{\Gamma \vdash \texttt{succ}\ n : \texttt{Nat}}\ \text{T-Succ} \qquad \frac{\Gamma \vdash n : \texttt{Nat} \qquad \Gamma \vdash z : \tau \qquad \Gamma \vdash s : \tau \to \tau}{\Gamma \vdash \texttt{iter}\ n\ z\ s : \tau}\ \text{T-Iter}$$

$$\frac{\Gamma \vdash t : \sigma \qquad \Gamma \vdash u : \tau}{\Gamma \vdash (t, u) : \sigma \times \tau}\ \text{T-Pair} \qquad \frac{\Gamma \vdash p : \sigma \times \tau}{\Gamma \vdash \texttt{fst}\ p : \sigma}\ \text{T-Fst} \qquad \frac{\Gamma \vdash p : \sigma \times \tau}{\Gamma \vdash \texttt{snd}\ p : \tau}\ \text{T-Snd}$$

$$\frac{\Gamma \vdash t : \sigma}{\Gamma \vdash \texttt{inl}\ t : \sigma + \tau}\ \text{T-Inl} \qquad \frac{\Gamma \vdash t : \tau}{\Gamma \vdash \texttt{inr}\ t : \sigma + \tau}\ \text{T-Inr}$$

$$\frac{\Gamma \vdash e : \sigma + \tau \qquad \Gamma, x : \sigma \vdash t : \rho \qquad \Gamma, y : \tau \vdash u : \rho}{\Gamma \vdash \texttt{case}\ e\ \texttt{of inl}\ x \Rightarrow t \mid \texttt{inr}\ y \Rightarrow u : \rho}\ \text{T-Case}$$

plus T-False (like T-True) and T-Unit, $\Gamma \vdash () : \texttt{Unit}$. Rule T-If is Chapter 1's rule, unchanged. Rule T-Case is Rust's `match`: both arms must produce the same type ρ, and each arm may use the payload of its variant. The table sums up the pattern, with the Rust feature that plays each role:

Type	Introduction	Elimination	In Rust
$\sigma \to \tau$	$\lambda x : \sigma.\, t$	application	closures, `fn`
$\sigma \times \tau$	(t, u)	`fst`, `snd`	tuples, `.0`, `.1`
$\sigma + \tau$	`inl`, `inr`	`case`	`enum`, `match`
`Bool`	`true`, `false`	`if`	`bool`, `if`
`Nat`	`0`, `succ`	`iter`	`u64`, loops, `fold`
`Unit`	`()`	(none needed)	`()`

If this table reminds you of Chapter 4, it should. Erase the terms from the rules and keep only the types: T-Lam becomes the introduction rule for implication, T-App is *modus ponens*, T-Pair and T-Fst are the rules for $\wedge$, T-Inl and T-Case the rules for $\vee$. That is not a coincidence, and Chapter 10 is devoted to it.

A derivation

Here is the derivation for `twice`, the function that applies its argument twice:

$$\vdash \lambda f : \texttt{Nat} \to \texttt{Nat}.\, \lambda x : \texttt{Nat}.\, f\,(f\,x) : (\texttt{Nat} \to \texttt{Nat}) \to \texttt{Nat} \to \texttt{Nat}$$

Write $\Gamma = f : \texttt{Nat} \to \texttt{Nat}, x : \texttt{Nat}$. The inner application $f\,x$ is typed by T-App from two variables:

$$\mathcal{D} = \frac{\dfrac{}{\Gamma \vdash f : \texttt{Nat} \to \texttt{Nat}}\ \text{T-Var} \qquad \dfrac{}{\Gamma \vdash x : \texttt{Nat}}\ \text{T-Var}}{\Gamma \vdash f\,x : \texttt{Nat}}\ \text{T-App}$$

The outer application uses $\mathcal{D}$ as its second premise, and two uses of T-Lam move x and then f out of the context and into the type:

$$\frac{\dfrac{\dfrac{\Gamma \vdash f : \texttt{Nat} \to \texttt{Nat} \qquad \mathcal{D}}{\Gamma \vdash f\,(f\,x) : \texttt{Nat}}\ \text{T-App}}{f : \texttt{Nat} \to \texttt{Nat} \vdash \lambda x : \texttt{Nat}.\, f\,(f\,x) : \texttt{Nat} \to \texttt{Nat}}\ \text{T-Lam}}{\vdash \lambda f : \texttt{Nat} \to \texttt{Nat}.\, \lambda x : \texttt{Nat}.\, f\,(f\,x) : (\texttt{Nat} \to \texttt{Nat}) \to \texttt{Nat} \to \texttt{Nat}}\ \text{T-Lam}$$

Notice that f is used twice. The context is shared by both premises of T-App, so a variable may be used as often as we like; that is contraction (Chapter 6) at work, and it is exactly what Part V will restrict.

8.3 Computation: β and η

The computation rules are the β-rule of Chapter 7 plus one rule for each elimination applied to an introduction:

- $(\lambda x : \tau.\, t)\ v \to t[x := v]$;
- $\texttt{fst}\ (v_1, v_2) \to v_1$ and $\texttt{snd}\ (v_1, v_2) \to v_2$;
- $\texttt{case}\ (\texttt{inl}\ v)\ \texttt{of inl}\ x \Rightarrow t \mid \texttt{inr}\ y \Rightarrow u \to t[x := v]$, and symmetrically for `inr`;
- $\texttt{if true then}\ t\ \texttt{else}\ u \to t$, and symmetrically for `false`;
- $\texttt{iter}\ 0\ z\ s \to z$ and $\texttt{iter}\ (\texttt{succ}\ n)\ z\ s \to s\ (\texttt{iter}\ n\ z\ s)$.

Each rule removes a *detour*: something was built only to be taken apart at once. Chapter 4 called the logical version of this **local soundness**, and you can now see why the slogan "proof simplification is computation" is literally true.

The **η-rules** go the other way. They say that every value of a type is determined by what you can do with it:

$$\lambda x : \sigma.\, f\ x \equiv f \quad (x \notin \mathrm{FV}(f)) \qquad\qquad (\texttt{fst}\ p, \texttt{snd}\ p) \equiv p$$

A function that only passes its argument on to f *is* f; a pair rebuilt from its two projections *is* the pair. Chapter 4 called this **local completeness**. η is usually stated as an equation rather than a reduction step, and it is the principle behind a refactoring you have probably done: Clippy's `redundant_closure` lint suggests replacing `.map(|x| f(x))` with `.map(f)`. Rust shows why η needs care in a real language, though. In `.map(|x| make_f()(x))`, `make_f()` runs once per element, while in `.map(make_f())` it runs once, before the call: under call-by-value with side effects, η-reducing can change when things happen.

8.4 What types buy: termination

The most striking consequence of the typing rules is a theorem.

> **Theorem (strong normalization).** If $\vdash t : \tau$ in the STLC, then every reduction sequence starting from t is finite.

Every well-typed program terminates, whatever the evaluation order. The theorem was proved by William Tait in the 1960s with a method now called **logical relations**, which Chapter 32 explains; for System T, the STLC with `iter`, the proof is a little longer but the result is the same. We will not prove it here, but we can see the most important thing it rules out: self-application.

Why $\lambda x.\, x\ x$ has no type

Suppose $\Gamma \vdash \lambda x : \sigma.\, x\ x : \tau$ for some annotation σ. Read the rules backward:

1. Only T-Lam concludes a judgment about a λ, so $\tau = \sigma \to \rho$ for some ρ, and $\Gamma, x : \sigma \vdash x\ x : \rho$.
2. Only T-App concludes a judgment about an application, so the first x has a type $\sigma' \to \rho$ and the second has type σ'.
3. Only T-Var types a variable, and it gives x the type σ in both places. So $\sigma = \sigma' \to \rho$ and $\sigma = \sigma'$.

Together: $\sigma = \sigma \to \rho$. But the type $\sigma \to \rho$ contains σ as a proper part, so it is strictly bigger than σ, and no type is equal to a strictly bigger type. There is no such σ. Without $\delta = \lambda x.\, x\, x$ there is no Ω, no Y and no Z.

This argument is short enough to check in Lean, and the file `code/lean/Book/Ch08/Extrinsic.lean` does. It defines raw, unannotated λ-terms with de Bruijn indices and a Curry-style typing relation `HasType`, one constructor per rule (we look at it more closely at the end of the chapter). First, the size argument:

```
/-- No type is a function type whose domain is the type itself: `σ ⇒ τ` is
always strictly bigger than `σ`. -/
theorem arrow_ne_self (σ τ : Ty) : (σ ⇒ τ) ≠ σ := by
  intro h
  -- Compare sizes: `sizeOf (σ ⇒ τ) = 1 + sizeOf σ + sizeOf τ`.
  have := congrArg sizeOf h
  simp at this
  omega
```

Every inductive type in Lean comes with a function `sizeOf` that counts constructors, and Lean proves equations such as `sizeOf (σ ⇒ τ) = 1 + sizeOf σ + sizeOf τ` automatically. `congrArg sizeOf h` applies `sizeOf` to both sides of the assumed equation `h : (σ ⇒ τ) = σ`; `simp` rewrites with the size equations, and `omega` (a decision procedure for linear arithmetic over the naturals) sees that `1 + sizeOf σ + sizeOf τ = sizeOf σ` is impossible. Then the theorem, which follows the three steps above exactly:

```
/-- `λx. x x` has no type, in any context. -/
theorem self_application_untypable (Γ : List Ty) (τ : Ty) :
    ¬ HasType Γ (.lam (.app (.var 0) (.var 0))) τ := by
  intro h
  -- Only rule `lam` concludes a judgment about a `λ`: τ = σ ⇒ ρ for some σ, ρ,
  -- and the body `x x` has type ρ when `x : σ`.
  cases h with
  | lam hbody =>
    -- Only rule `app` concludes a judgment about an application: the first
    -- `x` has some type σ' ⇒ ρ and the second has type σ'.
    cases hbody with
    | app hf ha =>
      -- Both are the variable 0, whose type is σ: so σ = σ' ⇒ ρ and σ = σ'.
      cases hf with
      | var e₁ =>
        cases ha with
        | var e₂ =>
          simp at e₁ e₂
          -- e₁ : σ = σ' ⇒ ρ and e₂ : σ = σ', hence σ' ⇒ ρ = σ'.
          subst e₂
          exact arrow_ne_self _ _ e₁.symm
```

Each `cases` is "only rule X concludes a judgment of this shape": Lean tries every constructor of `HasType`, discards those whose conclusion cannot match, and gives us the premises of the one that can. After the last two, `e₁` and `e₂` say what looking up index 0 in the context `σ :: Γ` returned; `simp` computes the lookup, leaving `σ = σ' ⇒ ρ` and `σ = σ'`, and `arrow_ne_self` finishes. Notice that the theorem holds for every context and every type: no annotation, no context, nothing can rescue self-application.

Rust agrees. Chapter 7 showed `|x| x(x)` failing because the type of `x` was unknown; giving `x` a type cannot help, by the argument above. Generics do not help either. Here a generic function asks for a closure `F` that can be called with a reference to its own type, and the closure we pass would need a type that contains itself:

```
fn self_apply<F: Fn(&F)>(f: F) {
    f(&f)
}

pub fn demo() {
    self_apply(|_me| println!("called"));
}
```

```
error[E0644]: closure/coroutine type that references itself
  --> compile_fail/ch08_closure_takes_itself.rs:11:16
   |
11 |     self_apply(|_me| println!("called"));
   |                ^^^^^ recursive type with infinite-size name
   |
   = note: closures cannot capture themselves or take themselves as argument;
           this error may be the result of a recent compiler bug-fix,
           see issue #46062 <https://github.com/rust-lang/rust/issues/46062>
           for more information
note: required by a bound in `self_apply`
  --> compile_fail/ch08_closure_takes_itself.rs:6:18
   |
 6 | fn self_apply<F: Fn(&F)>(f: F) {
   |                  ^^^^^^ required by this bound in `self_apply`
```

"Recursive type with infinite-size name" is the compiler's way of saying $\sigma = \sigma \to \rho$. (The note about a bug fix refers to the history of this check and can be ignored.)

The price of termination

Strong normalization is a strong guarantee, and it has a price: a language in which every program terminates cannot express every computable function. (A total language cannot even contain an interpreter for itself, by the same diagonal argument that makes the halting problem undecidable.) To get general recursion back, a language adds a fixed-point operator `fix` with the typing rule "if $f : \tau \to \tau$ then `fix` $f : \tau$", and loses termination. That extended calculus, usually called PCF, is the core of languages such as ML and Haskell.

Rust makes the same choice: recursive `fn`s and `loop` are unrestricted, so Rust is Turing complete, and its type system promises nothing about termination. A Rust function of type `fn(u64) -> u64` may run forever, and `loop {}` has every type (Chapter 1). Lean makes the opposite choice, and Chapter 7 showed what it costs: functions must be shown to terminate, or be marked `partial` and become opaque to proofs. Chapter 10 explains why a proof assistant has no alternative.

8.5 Bidirectional type checking

Writing a type on every λ is tedious, and Rust rarely asks you to. In `orders.iter().map(|o| o.total)` you do not annotate `o`; the compiler knows it from `map`'s signature. The type of the closure's parameter flows *into* the closure from its surroundings. **Bidirectional type checking** turns this observation into a disciplined algorithm by splitting the typing judgment in two:

- **synthesis**, $\Gamma \vdash t \Rightarrow \tau$: from the term alone, the checker computes its type τ (the type is an *output*);
- **checking**, $\Gamma \vdash t \Leftarrow \tau$: the checker is *given* τ and verifies that t has it (the type is an *input*).

Each rule is written for one mode. Variables, applications, projections and annotated terms synthesize; an unannotated λ is checked:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x \Rightarrow \tau}\ \text{S-Var} \qquad \frac{\Gamma \vdash t \Rightarrow \sigma \to \tau \qquad \Gamma \vdash u \Leftarrow \sigma}{\Gamma \vdash t\ u \Rightarrow \tau}\ \text{S-App} \qquad \frac{\Gamma \vdash t \Leftarrow \tau}{\Gamma \vdash (t : \tau) \Rightarrow \tau}\ \text{S-Ann}$$

$$\frac{\Gamma, x : \sigma \vdash t \Leftarrow \tau}{\Gamma \vdash \lambda x.\, t \Leftarrow \sigma \to \tau}\ \text{C-Lam} \qquad \frac{\Gamma \vdash t \Leftarrow \sigma}{\Gamma \vdash \texttt{inl}\ t \Leftarrow \sigma + \tau}\ \text{C-Inl} \qquad \frac{\Gamma \vdash t \Rightarrow \sigma \qquad \sigma = \tau}{\Gamma \vdash t \Leftarrow \tau}\ \text{C-Switch}$$

Read S-App as an algorithm: synthesize the type of the function; it must be an arrow $\sigma \to \tau$; now *check* the argument against σ, which we just learned; the result is τ. Read C-Lam the same way: we are given $\sigma \to \tau$, so we know the type of x without an annotation, and we check the body against τ. C-Switch connects the modes: to check a term that can synthesize, synthesize and compare. S-Ann goes the other way: an annotation turns a term that can only be checked into one that synthesizes.

The rules follow a simple recipe: **introduction forms check, elimination forms synthesize**. An elimination form has a "principal" subterm whose type determines everything (the function in an application, the pair in a projection), so it can synthesize. An introduction form often cannot: from $\lambda x.\, x$ alone there is no way to know the type of x, and from `inl` 3 there is no way to know the right half of the sum. That is why **a λ checks but does not synthesize**: the information it lacks, the type of its parameter, can only come from outside, from an expected type or an annotation. (A λ whose parameter *is* annotated can synthesize, and our checker allows that too.) In practice annotations are needed in only two places: at the top of a definition, and where a λ is applied on the spot, as in $(\lambda x.\, t)\ u$, which is rare in real code.

Bidirectional checking in Rust

Rust is not literally a bidirectional checker (Chapter 13 shows that it also solves equations between types), but its handling of closures has exactly this flavor. When a closure appears where a function type is expected, the expected type supplies its parameter types: checking mode. When it does not, the compiler must find the parameter type some other way, and if it needs it too early, it gives up:

```
pub fn total_length() -> usize {
    let len = |s| s.len();
    len("checked") + len("synthesized")
}
```

```
error[E0282]: type annotations needed
 --> compile_fail/ch08_closure_needs_annotation.rs:7:16
  |
7 |     let len = |s| s.len();
  |                ^  - type must be known at this point
  |
help: consider giving this closure parameter an explicit type
  |
7 |     let len = |s: /* Type */| s.len();
  |                 ++++++++++++
```

To call the method `len` the compiler must know the type of `s`, and at that point nothing has told it; the later call `len("checked")` comes too late. Two fixes correspond to the two rules above: annotate the parameter (`|s: &str| s.len()`, which makes the closure synthesize), or put the closure where a type is expected, as in `let len: fn(&str) -> usize = |s| s.len();` or `words.iter().map(|s| s.len())`

(checking mode, rule C-Lam). The same happens with sums: `let r = Ok(3);` alone is rejected with "type annotations needed for `Result<i32, _>`", because, like `inl` 3, `Ok(3)` says nothing about the error type.

8.6 The `stlc` crate

This chapter's crate, `code/rust/crates/stlc`, implements the language with a bidirectional checker and an evaluator. The syntax is two `enum`s:

```
/// Simple types: `τ ::= Unit | Bool | Nat | τ → τ | τ × τ | τ + τ`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Ty {
    Unit,
    Bool,
    Nat,
    /// Functions `σ → τ`.
    Fun(Box<Ty>, Box<Ty>),
    /// Pairs `σ × τ`.
    Prod(Box<Ty>, Box<Ty>),
    /// Tagged unions `σ + τ`: a value is `inl` of a `σ` or `inr` of a `τ`.
    Sum(Box<Ty>, Box<Ty>),
}
```

```
/// Terms. `Lam` may carry an annotation on its parameter; `Ann` is a type
/// annotation `(e : τ)` on any term.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    Var(String),
    /// `λx. e` or `λx: τ. e`
    Lam(String, Option<Ty>, Box<Term>),
    /// `e1 e2`
    App(Box<Term>, Box<Term>),
    /// `(e : τ)`
    Ann(Box<Term>, Ty),
    /// `let x = e1 in e2`
    Let(String, Box<Term>, Box<Term>),
    /// `()`
    Unit,
    /// `true`, `false`
    Bool(bool),
    /// `if c then e1 else e2`
    If(Box<Term>, Box<Term>, Box<Term>),
    /// `0`, `1`, `2`, ...: numerals stand for `succ (succ (... 0))`
    Nat(u64),
    /// `succ e`
    Succ(Box<Term>),
    /// `iter n z s`: apply `s` to `z`, `n` times
    Iter(Box<Term>, Box<Term>, Box<Term>),
    /// `(e1, e2)`
    Pair(Box<Term>, Box<Term>),
    /// `fst e`
    Fst(Box<Term>),
    /// `snd e`
    Snd(Box<Term>),
    /// `inl e`
    Inl(Box<Term>),
    /// `inr e`
    Inr(Box<Term>),
    /// `case e of inl x => e1 | inr y => e2`
    Case(Box<Term>, String, Box<Term>, String, Box<Term>),
}
```

`Lam` carries an `Option<Ty>`: `None` for $\lambda x.\, t$, `Some` for $\lambda x : \tau.\, t$. A parser and a printer handle the concrete syntax used in this chapter, with `λ`, `→` and `×` (or `\`, `->` and `*` in ASCII).

The checker

The checker is a struct holding the context, with two mutually recursive methods, one per judgment. Here are the synthesis rules for the λ-fragment:

```
Var(x) => self.lookup(x),
// An annotation switches to checking mode.
Ann(e, ty) => {
    self.check(e, ty)?;
    Ok(ty.clone())
}
// Elimination of →: synthesize the function's type, then check
// the argument against its domain.
App(f, a) => match self.infer(f)? {
    Ty::Fun(dom, cod) => {
        self.check(a, &dom)?;
        Ok(*cod)
    }
    found => Err(TypeError::NotAFunction {
        term: (**f).clone(),
        found,
    }),
},
// A λ with an annotated parameter can synthesize...
Lam(x, Some(dom), body) => {
    let cod = self.under(x, dom.clone(), |c| c.infer(body))?;
    Ok(fun(dom.clone(), cod))
}
// ...but without one it cannot: what is the type of x?
Lam(_, None, _) => Err(TypeError::CannotSynthesize {
    term: e.clone(),
    form: "an unannotated λ",
}),
```

Each arm is one rule: `Var` is S-Var, `Ann` is S-Ann (check, then return the annotation), `App` is S-App, an annotated `Lam` synthesizes by extending the context and synthesizing the body, and an unannotated `Lam` is an error. The rules for the other forms follow the recipe: literals synthesize, `if` synthesizes the type of its first branch and checks the second against it, and `inl` / `inr` refuse to synthesize. The checking rules:

```
/// The checking rules, `Γ ⊢ e ⇐ τ`: does `e` have the given type `τ`? The
/// type is an input.
fn check_inner(&mut self, e: &Term, ty: &Ty) -> Result<(), TypeError> {
    use Term::*;
    let wrong_shape = |form| TypeError::WrongShape {
        term: e.clone(),
        expected: ty.clone(),
        form,
    };
    match (e, ty) {
        // Introduction of →: the expected type tells us the type of x.
        (Lam(x, ann, body), Ty::Fun(dom, cod)) => {
            if let Some(ann) = ann
                && ann != &**dom
            {
                return Err(TypeError::Mismatch {
                    term: e.clone(),
                    expected: ty.clone(),
                    found: fun(ann.clone(), (**cod).clone()),
                });
            }
            self.under(x, (**dom).clone(), |c| c.check(body, cod))
        }
        (Lam(..), _) => Err(wrong_shape("a function")),
        (Pair(a, b), Ty::Prod(ta, tb)) => {
            self.check(a, ta)?;
```

```
            self.check(b, tb)
        }
        (Pair(..), _) => Err(wrong_shape("a pair")),
        (Inl(a), Ty::Sum(left, _)) => self.check(a, left),
        (Inr(b), Ty::Sum(_, right)) => self.check(b, right),
        (Inl(_) | Inr(_), _) => Err(wrong_shape("an injection into a sum")),
        (If(c, a, b), _) => {
            self.check(c, &Ty::Bool)?;
            self.check(a, ty)?;
            self.check(b, ty)
        }
        (Let(x, e1, e2), _) => {
            let ty1 = self.infer(e1)?;
            self.under(x, ty1, |c| c.check(e2, ty))
        }
        (Case(scrut, x, l, y, r), _) => {
            let (left, right) = self.infer_sum(scrut)?;
            self.under(x, left, |c| c.check(l, ty))?;
            self.under(y, right, |c| c.check(r, ty))
        }
        (Iter(n, z, s), _) => {
            self.check(n, &Ty::Nat)?;
            self.check(z, ty)?;
            self.check(s, &fun(ty.clone(), ty.clone()))
        }
        // Everything else: synthesize a type and compare (rule Switch).
        _ => {
            let found = self.infer(e)?;
            if &found == ty {
                Ok(())
            } else {
                Err(TypeError::Mismatch {
                    term: e.clone(),
                    expected: ty.clone(),
                    found,
                })
            }
        }
    }
}
```

The first arm is C-Lam: the expected type $\sigma \to \tau$ provides the parameter's type (and if there is an annotation too, it must agree). Pairs, injections, `if`, `let`, `case` and `iter` push the expected type into their parts. The last arm is C-Switch. The method `under` extends the context for the duration of a closure and pops the binding afterwards, so that inner bindings shadow outer ones and disappear when their scope ends.

The crate can record every judgment the checker visits. The example `crates/stlc/examples/ch08_stlc.rs` checks an unannotated `twice` against $(\texttt{Nat} \to \texttt{Nat}) \to \texttt{Nat} \to \texttt{Nat}$:

```
Checking λf. λx. f (f x) against (Nat → Nat) → Nat → Nat:
  ⊢ λf. λx. f (f x) ⇐ (Nat → Nat) → Nat → Nat
    f: Nat → Nat ⊢ λx. f (f x) ⇐ Nat → Nat
      f: Nat → Nat, x: Nat ⊢ f (f x) ⇐ Nat
        f: Nat → Nat, x: Nat ⊢ f (f x) ⇒ Nat
          f: Nat → Nat, x: Nat ⊢ f ⇒ Nat → Nat
          f: Nat → Nat, x: Nat ⊢ f x ⇐ Nat
            f: Nat → Nat, x: Nat ⊢ f x ⇒ Nat
              f: Nat → Nat, x: Nat ⊢ f ⇒ Nat → Nat
              f: Nat → Nat, x: Nat ⊢ x ⇐ Nat
                f: Nat → Nat, x: Nat ⊢ x ⇒ Nat
```

Read it from the top. Two C-Lam steps move f and x into the context, with types taken from the expected type. The body `f (f x)` is checked against `Nat`, but an application cannot be checked directly:

C-Switch synthesizes its type instead (the `⇒` line one level down, with the result `Nat`, compared with the expected `Nat`). S-App synthesizes the type of `f`, `Nat → Nat`, and checks the argument `f x` against `Nat`, which repeats the pattern one level deeper, ending with the variable `x`, checked by switching to synthesis. Information flows down through `⇐` and back up through `⇒`, and no annotation was needed.

When the search for a derivation fails, the error points at the subterm where it failed:

```
Rejected by the checker:
  3 4
      `3` has type Nat; it is not a function
  λx. x
      cannot synthesize a type for `λx. x` (an unannotated λ can only be checked)
  if true then 1 else false
      `false` has type Bool, but Nat was expected
  (λx. x : Nat)
      `λx. x` is a function, but Nat was expected
  λx: Nat → Nat. x x
      `x` has type Nat → Nat, but Nat was expected
```

The last one is self-application with an annotation: whatever type we give `x`, the second `x` is checked against the domain of the first, and the types do not match.

The evaluator: environments and closures

To *run* programs, the crate does not substitute, as Chapter 7 did; it uses an **environment**, a list of variable bindings, the way real interpreters and compilers work. Evaluating a λ cannot evaluate its body (the parameter has no value yet), so it produces a **closure**: the parameter, the body, and the values of the body's other free variables, captured from the environment.

```rust
/// The results of evaluation.
#[derive(Clone, Debug, PartialEq)]
pub enum Value {
    Unit,
    Bool(bool),
    Nat(u64),
    Pair(Rc<Value>, Rc<Value>),
    Inl(Rc<Value>),
    Inr(Rc<Value>),
    Closure(Rc<Closure>),
}

/// A function value: a parameter, a body, and the captured variables.
#[derive(Debug, PartialEq)]
pub struct Closure {
    pub param: String,
    pub body: Term,
    /// The values of the body's free variables (other than the parameter),
    /// copied from the environment when the closure was created.
    pub captured: Env,
}
```

```rust
/// Builds the closure for `λparam. body` in `env`, capturing only the free
/// variables of the λ, as a Rust closure does.
fn make_closure(env: &Env, param: &str, body: &Term) -> Result<Value, Stuck> {
    let lam = Term::Lam(param.to_string(), None, Box::new(body.clone()));
    let mut captured = Vec::new();
    for x in lam.free_vars() {
        captured.push((x.clone(), lookup(env, &x)?));
    }
    Ok(Value::Closure(Rc::new(Closure {
        param: param.to_string(),
        body: body.clone(),
```

```
        captured,
    }))
}
```

```
/// Calls a function value: evaluate the body in the captured environment
/// extended with the argument.
pub fn apply(f: &Value, arg: Value) -> Result<Value, Stuck> {
    match f {
        Value::Closure(c) => eval(&extend(&c.captured, &c.param, arg), &c.body),
        other => Err(Stuck(format!("cannot call {other}: it is not a function"))),
    }
}
```

`make_closure` computes the free variables of the λ (Chapter 7) and copies their values, and only theirs, out of the environment. Calling a closure evaluates its body in the captured environment extended with the argument. This is **lexical scope**: a function sees the variables of the place where it was *defined*, not of the place where it is called. The evaluator itself is call-by-value, like Rust:

```
/// Evaluates a term in an environment, call-by-value.
pub fn eval(env: &Env, e: &Term) -> Result<Value, Stuck> {
    use Term::*;
    match e {
        Var(x) => lookup(env, x),
        Lam(x, _, body) => make_closure(env, x, body),
        App(f, a) => {
            let f = eval(env, f)?;
            let a = eval(env, a)?; // call-by-value: the argument first
            apply(&f, a)
        }
        Ann(e, _) => eval(env, e),
        Let(x, e1, e2) => {
            let v = eval(env, e1)?;
            eval(&extend(env, x, v), e2)
        }
        Unit => Ok(Value::Unit),
        Bool(b) => Ok(Value::Bool(*b)),
        If(c, a, b) => match eval(env, c)? {
            Value::Bool(true) => eval(env, a),
            Value::Bool(false) => eval(env, b),
            other => Err(Stuck(format!("`if` on {other}, which is not a boolean"))),
        },
        Nat(n) => Ok(Value::Nat(*n)),
        Succ(n) => match eval(env, n)? {
            Value::Nat(k) => Ok(Value::Nat(k + 1)),
            other => Err(Stuck(format!("`succ` of {other}, which is not a number"))),
        },
        Iter(n, z, s) => match eval(env, n)? {
            Value::Nat(k) => {
                let s = eval(env, s)?;
                let mut acc = eval(env, z)?;
                for _ in 0..k {
                    acc = apply(&s, acc)?;
                }
                Ok(acc)
            }
            other => Err(Stuck(format!("`iter` on {other}, which is not a number"))),
        },
        Pair(a, b) => Ok(Value::Pair(Rc::new(eval(env, a)?), Rc::new(eval(env, b)?))),
        Fst(p) | Snd(p) => match eval(env, p)? {
            Value::Pair(a, b) => {
                let v = if matches!(e, Fst(_)) { a } else { b };
                Ok((*v).clone())
            }
            other => Err(Stuck(format!("projection of {other}, which is not a pair"))),
        },
        Inl(a) => Ok(Value::Inl(Rc::new(eval(env, a)?))),
```

```
        Inr(b) => Ok(Value::Inr(Rc::new(eval(env, b)?))),
        Case(scrut, x, l, y, r) => match eval(env, scrut)? {
            Value::Inl(v) => eval(&extend(env, x, (*v).clone()), l),
            Value::Inr(v) => eval(&extend(env, y, (*v).clone()), r),
            other => Err(Stuck(format!("`case` on {other}, which is not a sum"))),
        },
    }
}
```

Here is the crate's small library, with the types the checker synthesizes, and some programs that use it (`with_library` wraps each program in `let`s that define the library):

```
The library, with synthesized types:
  twice  : (Nat → Nat) → Nat → Nat
  add    : Nat → Nat → Nat
  mul    : Nat → Nat → Nat
  iszero : Nat → Bool
  pred   : Nat → Nat
  swap   : Nat × Bool → Bool × Nat
```

```
Evaluation:
  twice (add 3) 4  :  Nat  ⇓  10
  mul 6 7  :  Nat  ⇓  42
  pred 10  :  Nat  ⇓  9
  iszero 0  :  Bool  ⇓  true
  swap (7, true)  :  Bool × Nat  ⇓  (true, 7)
  case (inr true : Nat + Bool) of inl n => n | inr b => 1  :  Nat  ⇓  1
  add 3  :  Nat → Nat  ⇓  <closure λn. iter m n (λk: Nat. succ k) | m = 3>
```

`pred` is Kleene's pair trick from Chapter 7, now typed. The last line is the interesting one: `add 3` is a function, and its value is a closure whose body is `λn. iter m n (λk: Nat. succ k)` and whose captured environment is `m = 3`. Only `m` is captured, because it is the only free variable of the body; the library functions in scope are not.

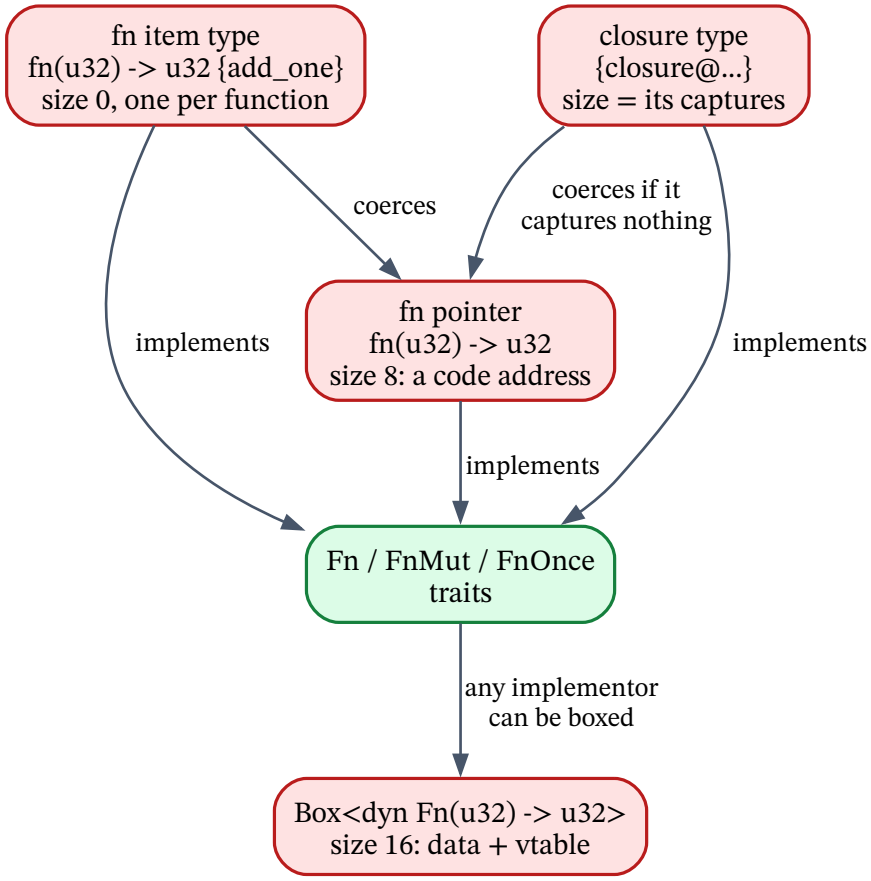
Look at the `Err(Stuck(...))` arms of `eval`. They are reached when the evaluator finds something it cannot handle, such as an `if` on a number. If we skip the checker, ill-typed programs reach them:

```
Run without checking:
  3 4  ⇓  stuck: cannot call 3: it is not a function
  if 0 then 1 else 2  ⇓  stuck: `if` on 0, which is not a boolean
```

For every program the checker accepts, these arms are dead code. That claim is the **progress** theorem, and Chapter 9 proves it, for a language like this one, in Lean.

8.7 Functions in Rust

In the STLC there is one kind of function type, $\sigma \to \tau$. In Rust there are several, because Rust cares about two things the calculus ignores: what a function value occupies in memory, and what it is allowed to do with the data it captured. The example `code/rust/examples/ch08_closures.rs` explores them all.



Rust's function types and how they relate: fn items and non-capturing closures coerce to fn pointers, and all of them implement the Fn traits, through which any of them can be boxed as a trait object.

`fn` items and function pointers

A function declared with `fn` is called a **`fn` item**, and each one has its own type, distinct from every other function's, even one with the same signature:

```
fn add_one(x: u32) -> u32 {
    x + 1
}

fn double(x: u32) -> u32 {
    x * 2
}
```

```
let f = add_one;
println!(
    "add_one: type {}, size {}",
    type_name_of_val(&f),
    size_of_val(&f)
);
let table: [fn(u32) -> u32; 2] = [add_one, double];
let p = table[1];
println!(
    "table[1]: type {}, size {}",
    type_name_of_val(&p),
    size_of_val(&p)
);
println!("table[1](21) = {}", p(21));
```

```
add_one: type ch08_closures::add_one, size 0
table[1]: type fn(u32) -> u32, size 8
table[1](21) = 42
```

The type of `add_one` is a **zero-sized type**: the compiler knows statically which function it is, so a value of that type needs no bytes at all, and a call through it compiles to a direct call. (The compiler writes this type as `fn(u32) -> u32 {add_one}` in error messages.) To put two different functions in one array, they must have one type, and Rust **coerces** both to the **function pointer** type `fn(u32) -> u32`: an 8-byte code address, called indirectly. A function pointer is the closest thing in Rust to the $\sigma \to \tau$ of the STLC: one type for all functions with that signature.

Closures and their anonymous types

A closure is more than code: it carries the values of the variables it captures. The compiler implements each closure expression as an anonymous struct with one field per capture, plus an implementation of the call operator. Roughly (this sketch is not compiled; the real traits cannot be implemented by hand in stable Rust):

```
// let percent: u16 = 3;
// let by_value = move |x: u16| x * percent;
// becomes, approximately:
struct ByValueClosure {
    percent: u16, // the captured variable
}
impl Fn(u16) -> u16 for ByValueClosure {
    fn call(&self, x: u16) -> u16 {
        x * self.percent
    }
}
```

This is the same representation as the `stlc` crate's `Closure`: code plus captured environment. The size of a closure is the size of its captures:

```
let percent: u16 = 3;
let label = String::from("invoice 2041");
let nothing = |x: u16| x + 1;
let by_ref = |x: u16| x * percent; // captures &percent: a pointer
let by_value = move |x: u16| x * percent; // captures a copy of percent
let owns_label = move || label.len(); // captures the String itself
println!("closure type: {}", type_name_of_val(&nothing));
println!(
    "sizes: nothing {}, by_ref {}, by_value {}, owns_label {}",
    size_of_val(&nothing),
    size_of_val(&by_ref),
    size_of_val(&by_value),
    size_of_val(&owns_label)
);
assert_eq!(
    (nothing(1), by_ref(1), by_value(1), owns_label()),
    (2, 3, 3, 12)
);
```

```
closure type: ch08_closures::main::{{closure}}
sizes: nothing 0, by_ref 8, by_value 2, owns_label 24
```

A closure that captures nothing is zero-sized, like a `fn` item. One that captures `percent` by reference holds a pointer (8 bytes); one that captures it by value holds a copy of the `u16` (2 bytes); one that owns a `String` holds the `String` itself (24 bytes: pointer, capacity and length).

Since each closure expression defines its own struct, **every closure has its own type**, and no two closures, even textually identical ones, have the same type. The type has no name you can write, which is why the compiler prints it as `{closure@...}` with a source location. This runs into a rule you know from Chapter 1, T-If:

```
pub fn scaler(double: bool, k: u32) -> impl Fn(u32) -> u32 {
    let times_k = move |x| x * k;
    let plus_k = move |x| x + k;
    if double { times_k } else { plus_k }
}
```

```
error[E0308]: `if` and `else` have incompatible types
 --> compile_fail/ch08_closure_types_differ.rs:9:34
  |
7 |     let times_k = move |x| x * k;
  |                   -------- the expected closure
8 |     let plus_k = move |x| x + k;
  |                  -------- the found closure
9 |     if double { times_k } else { plus_k }
  |                 -------          ^^^^^^ expected closure, found a different closure
  |                 |
  |                 expected because of this
  |
  = note: expected closure `{closure@compile_fail/ch08_closure_types_differ.rs:7:19: 7:27}`
             found closure `{closure@compile_fail/ch08_closure_types_differ.rs:8:18: 8:26}`
  = note: no two closures, even if identical, have the same type
  = help: consider boxing your closure and/or using it as a trait object
help: you could change the return type to be a boxed trait object
  |
6 - pub fn scaler(double: bool, k: u32) -> impl Fn(u32) -> u32 {
6 + pub fn scaler(double: bool, k: u32) -> Box<dyn Fn(u32) -> u32> {
  |
help: if you change the return type to expect trait objects, box the returned expressions
  |
9 |     if double { Box::new(times_k) } else { Box::new(plus_k) }
  |                 +++++++++       +          +++++++++      +
```

The two branches have two different types, so no instance of T-If applies. The compiler's suggestion is the standard fix: erase the difference behind a pointer, `Box<dyn Fn(u32) -> u32>`. (If neither closure captured anything, the program would compile: both would be coerced to the function pointer type `fn(u32) -> u32`.) A capturing closure can never become a function pointer, because a code address has nowhere to keep the captured data:

```
pub fn shift_all(xs: &mut [u32], offset: u32) {
    let shift: fn(u32) -> u32 = |x| x + offset;
    for x in xs {
        *x = shift(*x);
    }
}
```

```
error[E0308]: mismatched types
 --> compile_fail/ch08_capture_to_fn_pointer.rs:7:33
  |
7 |     let shift: fn(u32) -> u32 = |x| x + offset;
  |                ---------------   ^^^^^^^^^^^^^^^ expected fn pointer, found closure
  |                |
```

```
   |                expected due to this
   |
   = note: expected fn pointer `fn(u32) -> u32`
                 found closure `{closure@compile_fail/ch08_capture_to_fn_pointer.rs:7:33:
7:36}`
note: closures can only be coerced to `fn` types if they do not capture any variables
  --> compile_fail/ch08_capture_to_fn_pointer.rs:7:41
   |
7  |     let shift: fn(u32) -> u32 = |x| x + offset;
   |                                         ^^^^^^ `offset` captured here
```

`impl Fn` and `dyn Fn`

Since closure types cannot be named, functions that accept or return closures name them through the traits they implement. `impl Fn(u64) -> u64` in argument position means "some type implementing this trait, chosen by the caller": it is a generic parameter in disguise, and each call is compiled separately for the closure's actual type (monomorphization, Chapter 12), so the call is direct and can be inlined. In return position it means "some type chosen by the function, which the caller may not know": an *existential* type, the subject of Chapter 15. `dyn Fn(u64) -> u64` is a **trait object**: a pointer to the closure's data together with a pointer to a table of its methods (the *vtable*). It is the only way to treat closures of different types uniformly, for example in a `Vec`:

```rust
// Different closures have different types; to store them together we
// erase the types behind a pointer: `Box<dyn Fn>`.
let discount = 10;
let add_fee = fee_curried(3);
let pipeline: Vec<Box<dyn Fn(u64) -> u64>> = vec![
    Box::new(move |c| c - c * discount / 100), // 10% off
    Box::new(move |c| c + add_fee(c)),         // plus a 3% fee
    Box::new(|c| c / 100 * 100),               // whole units only
];
let result = pipeline.iter().fold(25_000, |amount, step| step(amount));
println!("pipeline(25000) = {result}");
println!(
    "size of Box<dyn Fn(u64) -> u64> = {}",
    size_of_val(&pipeline[0])
);
```

```
pipeline(25000) = 23100
size of Box<dyn Fn(u64) -> u64> = 16
```

The box costs a heap allocation (unless the closure is zero-sized), and each call goes through the vtable, an indirect call the compiler usually cannot inline. The three representations, generic `impl Fn`, function pointer and `dyn Fn`, are three engineering answers to one mathematical type $\sigma \to \tau$.

Capture modes

How does a closure capture a variable? The compiler picks the least powerful mode that the body needs:

- **by shared reference** (`&x`) if the body only reads `x`;
- **by mutable reference** (`&mut x`) if the body modifies `x`;
- **by value** (moving `x` into the closure) if the body moves `x` out.

The keyword `move` overrides the choice: every captured variable is captured by value (moved, or copied if its type is `Copy`). Since the 2021 edition, a closure that uses only a field `x.a` captures just that field, not all of `x`.

```
// By shared reference: the closure only reads `prices`.
let prices = vec![120, 80, 45];
let total = || prices.iter().sum::<u32>();
println!(
    "total = {}, and prices is still usable: {:?}",
    total(),
    prices
);

// By mutable reference: the closure updates `calls`.
let mut calls = 0;
let mut count = || {
    calls += 1;
    calls
};
count();
count();
println!("calls = {calls}");

// By value: the closure moves `report` out of itself when called.
let report = String::from("Q3 report");
let send = move || report;
println!("sent: {}", call_once(send));
```

```
total = 245, and prices is still usable: [120, 80, 45]
calls = 2
sent: Q3 report
```

`total` borrows `prices`, so `prices` stays usable. `count` borrows `calls` mutably for as long as `count` is in use, so `calls` can be printed only after the last call. `send` owns `report`. Borrowing captures are governed by the borrow checker like any other reference, and the most common surprise is a closure that outlives what it borrows:

```
pub fn make_adder(n: u32) -> impl Fn(u32) -> u32 {
    |x| x + n
}
```

```
error[E0373]: closure may outlive the current function, but it borrows `n`, which is owned
by the current function
 --> compile_fail/ch08_closure_escapes.rs:7:5
  |
7 |     |x| x + n
  |     ^^^     - `n` is borrowed here
  |     |
  |     may outlive borrowed value `n`
  |
note: closure is returned here
 --> compile_fail/ch08_closure_escapes.rs:7:5
  |
7 |     |x| x + n
  |     ^^^^^^^^^
help: to force the closure to take ownership of `n` (and any other referenced variables),
use the `move` keyword
  |
7 |     move |x| x + n
  |     ++++
```

The closure reads `n`, so it captures `&n`, a reference to a local variable that dies when `make_adder` returns. `move` fixes it by storing a copy of `n` in the closure. Lifetimes of references are the subject of Part VI; here it is enough to see that the λ-calculus has no such issue, because its variables are not places in memory.

`Fn`, `FnMut` and `FnOnce`

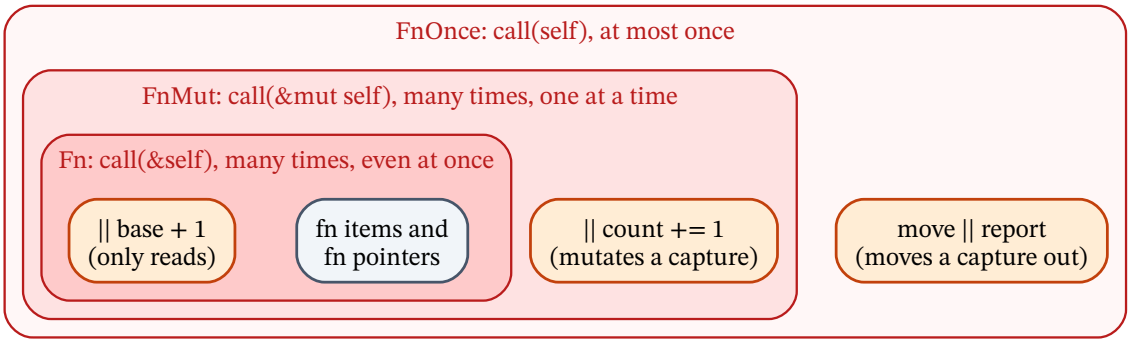
Calling a closure is calling a method on its struct, and the question is how the method receives `self`. The three possibilities are three traits, here in simplified form (the real ones are generic over the argument tuple and cannot be implemented in stable Rust):

```
trait FnOnce<Args> {
    type Output;
    fn call_once(self, args: Args) -> Self::Output; // consumes the closure
}
trait FnMut<Args>: FnOnce<Args> {
    fn call_mut(&mut self, args: Args) -> Self::Output; // exclusive access
}
trait Fn<Args>: FnMut<Args> {
    fn call(&self, args: Args) -> Self::Output; // shared access
}
```

What a closure's body does with its captures decides which traits it implements:

- if it moves a captured value out, it can only be called by value, once: it implements `FnOnce` only;
- if it mutates a capture, it needs exclusive access: `FnMut` (and therefore `FnOnce`);
- if it only reads, shared access suffices: `Fn` (and therefore `FnMut` and `FnOnce`).

The supertrait chain `Fn: FnMut: FnOnce` says that every `Fn` closure can be used where an `FnMut` or `FnOnce` is expected: a function that may be called many times through a shared reference may certainly be called through an exclusive one, or once. `fn` items and function pointers implement all three.



The three closure traits as nested sets: every Fn closure is also FnMut, and every FnMut closure is also FnOnce.

Functions that *accept* closures choose the weakest trait they need, which gives the caller the most freedom:

```
/// Calls `f` twice. `Fn` means: callable through a shared reference.
fn call_twice(f: impl Fn() -> u32) -> u32 {
    f() + f()
```

```
}

/// Calls `f` twice. `FnMut` means: callable through a mutable reference.
fn call_twice_mut(mut f: impl FnMut() -> u32) -> u32 {
    f() + f()
}

/// Calls `f` once. `FnOnce` means: callable by value, consuming it.
fn call_once(f: impl FnOnce() -> String) -> String {
    f()
}
```

```
let base = 20;
let reads = || base + 1; // Fn: only reads its capture
println!("call_twice(reads)     = {}", call_twice(reads));
println!("call_twice_mut(reads) = {}", call_twice_mut(reads));
let mut ticks = 0;
let bumps = || {
    ticks += 10;
    ticks
}; // FnMut: mutates its capture
println!("call_twice_mut(bumps) = {}", call_twice_mut(bumps));
// `move` decides HOW variables are captured, not WHICH trait you get:
let greeting = String::from("hello");
let greet = move || greeting.len() as u32; // moved in, but only read
println!("call_twice(greet)     = {}", call_twice(&greet));
println!("greet is still callable: {}", greet());
```

```
call_twice(reads)     = 42
call_twice_mut(reads) = 42
call_twice_mut(bumps) = 30
call_twice(greet)     = 10
greet is still callable: 5
```

`reads` is `Fn`, so it can be passed to both `call_twice` and `call_twice_mut`. `bumps` mutates `ticks`, so it is only `FnMut`. `greet` was declared `move`, but its body only reads `greeting`, so it is `Fn`: **`move` decides how variables are captured, not which traits the closure implements**. Passing `&greet` works because a shared reference to an `Fn` closure is itself an `Fn` closure, and afterwards `greet` is still ours. The compiler rejects the two combinations that would be unsound. A mutating closure where `Fn` is required:

```
fn call_twice<F: Fn()>(f: F) {
    f();
    f();
}

pub fn count_calls() {
    let mut count = 0;
    call_twice(|| count += 1);
}
```

```
error[E0594]: cannot assign to `count`, as it is a captured variable in a `Fn` closure
  --> compile_fail/ch08_fn_mutates.rs:13:19
   |
 6 | fn call_twice<F: Fn()>(f: F) {
   |                        - change this to accept `FnMut` instead of `Fn`
...
12 |     let mut count = 0;
   |         --------- `count` declared here, outside the closure
13 |     call_twice(|| count += 1);
   |     ---------- -- ^^^^^^^^^^ cannot assign
   |     |          |
```

```
   |     |          in this closure
   |     expects `Fn` instead of `FnMut`
```

and an `FnOnce` closure called twice:

```
pub fn send_twice() {
    let report = String::from("Q3 report");
    let send = move || report; // returns `report`, moving it out
    let first = send();
    let second = send();
}
```

```
error[E0382]: use of moved value: `send`
  --> compile_fail/ch08_fnonce_twice.rs:10:18
   |
 9 |     let first = send();
   |                 ------ `send` moved due to this call
10 |     let second = send();
   |                  ^^^^ value used here after move
   |
note: closure cannot be invoked more than once because it moves the variable `report` out of
its environment
  --> compile_fail/ch08_fnonce_twice.rs:8:24
   |
 8 |     let send = move || report; // returns `report`, moving it out
   |                        ^^^^^^
note: this value implements `FnOnce`, which causes it to be moved when called
  --> compile_fail/ch08_fnonce_twice.rs:9:17
   |
 9 |     let first = send();
   |                 ^^^^
```

The second error is E0382, "use of moved value", the error of Chapter 6. The first call moved `send` (calling an `FnOnce` consumes it), so there is nothing left to call. Read the three traits as answers to the question **how many times may this function be used?** `FnOnce` : at most once, because a call uses up resources. `FnMut` : any number of times, one at a time. `Fn` : any number of times, even simultaneously from several places. In the STLC the question does not arise: every function is a value that can be used any number of times, because the calculus has contraction. Rust's `FnOnce` is a function that can be used *at most once*, an **affine** function, and Chapter 21 will write its type as $A \multimap B$, the linear implication of linear logic. This is the first place where the book's question, whether Rust's ownership is "based on affine types", shows up in the type of a function.

The cost of currying

In the λ-calculus, a two-argument function and its curried form are interchangeable at no cost: $(A \times B) \to C$ and $A \to (B \to C)$ are isomorphic (Chapter 11 proves it), and partial application is just application. In Rust, currying a particular function by hand is easy and cheap:

```
/// A fee of `percent` percent on `amount` cents.
fn fee(percent: u64, amount: u64) -> u64 {
    amount * percent / 100
}

/// The same function, curried by hand: the result captures `percent`.
fn fee_curried(percent: u64) -> impl Fn(u64) -> u64 {
    move |amount| fee(percent, amount)
}
```

`fee_curried(3)` is a closure that captures `percent`: 8 bytes, no allocation. But a *generic* `curry`, turning any two-argument function into a curried one, runs into three costs at once. The natural signature does not even compile:

```
pub fn curry<A: Clone, B, C>(
    f: impl Fn(A, B) -> C + Clone,
) -> impl Fn(A) -> impl Fn(B) -> C {
    move |a: A| {
        let f = f.clone();
        move |b: B| f(a.clone(), b)
    }
}
```

```
error[E0562]: `impl Trait` is not allowed in the return type of `Fn` trait bounds
 --> compile_fail/ch08_curry_nested_impl.rs:8:20
  |
8 | ) -> impl Fn(A) -> impl Fn(B) -> C {
  |                     ^^^^^^^^^^^^^^
  |
  = note: `impl Trait` is only allowed in arguments and return types of functions and
methods
  = note: see issue #99697 <https://github.com/rust-lang/rust/issues/99697> for more
information
```

Rust allows `impl Trait` in the return type of a function, but not inside the return type of an `Fn` bound. The version that compiles boxes the inner closure:

```
/// Currying, for any two-argument function. The inner closure is returned as
/// a `Box<dyn Fn>` (Rust does not allow `impl Fn(A) -> impl Fn(B) -> C`), and
/// it must clone `a` on every call, because it may be called many times.
fn curry<A, B, C>(
    f: impl Fn(A, B) -> C + Clone + 'static,
) -> impl Fn(A) -> Box<dyn Fn(B) -> C>
where
    A: Clone + 'static,
    B: 'static,
    C: 'static,
{
    move |a: A| {
        let f = f.clone();
        Box::new(move |b: B| f(a.clone(), b))
    }
}
```

1. **An allocation.** The inner closure is a `Box<dyn Fn>`: one heap allocation per partial application (it holds at least `a`), and an indirect call per use.
2. **Clones.** The inner closure is `Fn`, so it may be called many times, and each call needs its own `A` to pass to `f`, which takes `A` by value. It must clone `a` on every call. The outer closure must also clone `f` into each inner closure, hence `Clone` on `f`.
3. **`'static` bounds.** A boxed trait object must say how long the data it points to lives; here we require it to own everything (`'static`, Chapter 26).

The example counts the clones with a type that increments a counter in its `Clone` implementation:

```
println!("fee(3, 2000)            = {}", fee(3, 2000));
println!("fee_curried(3)(2000)    = {}", fee_curried(3)(2000));
println!("curry(fee)(3)(2000)     = {}", curry(fee)(3)(2000));
println!("size of fee_curried(3)  = {}", size_of_val(&fee_curried(3)));
println!(
    "size of curry(fee)(3)   = {} (plus a heap allocation)",
```

```
    size_of_val(&curry(fee)(3))
);

let tag = |t: Tracked, line: &str| format!("[{}] {line}", t.0);
let tagger = curry(tag)(Tracked("audit".to_string()));
for line in ["opened", "approved", "closed"] {
    println!("{}", tagger(line));
}
println!(
    "clones of the captured Tracked: {}",
    CLONES.load(Ordering::Relaxed)
);
```

```
fee(3, 2000)            = 60
fee_curried(3)(2000)    = 60
curry(fee)(3)(2000)     = 60
size of fee_curried(3)  = 8
size of curry(fee)(3)   = 16 (plus a heap allocation)
[audit] opened
[audit] approved
[audit] closed
clones of the captured Tracked: 3
```

Three calls, three clones of the captured value. None of this is a flaw in Rust: it is what currying *means* once values have owners and functions may be called many times. The λ-calculus duplicates freely (contraction) and never frees anything; Rust makes duplication explicit and memory management visible, and the cost of currying is the bill.

8.8 Intrinsically typed terms in Lean

So far, a term and its type have been separate things: there is a set of terms, some of which are well typed, and a typing relation that says which. This is called **extrinsic** typing ("types from outside"), and it is the style of Chapter 3, of the `stlc` crate, and of Chapter 9. Lean, with its dependent types, allows another style. In **intrinsic** typing, the type of a term is part of the term's *Lean type*: `Term Γ τ` is the type of terms of type `τ` in context `Γ`, and a term that is not well typed cannot be constructed at all. The file is `code/lean/Book/Ch08/Intrinsic.lean`.

Types, and what they mean

The object language's types are an ordinary inductive type, with a scoped notation `⇒` for the arrow (scoped so that it does not leak into other files), and each type has a *meaning*: the Lean type of its values.

```
/-- The simple types: unit, booleans, naturals, functions, pairs and sums. -/
inductive Ty where
  | unit
  | bool
  | nat
  | arrow (σ τ : Ty)
  | prod (σ τ : Ty)
  | sum (σ τ : Ty)
  deriving DecidableEq, Repr

/-- `σ ⇒ τ` is the function type (scoped: it does not leak into other files). -/
scoped infixr:25 " ⇒ " => Ty.arrow
```

```
/-- What a type means: the Lean type of its values. -/
@[reducible] def Ty.denote : Ty → Type
  | unit => Unit
  | bool => Bool
  | nat => Nat
  | arrow σ τ => σ.denote → τ.denote
  | prod σ τ => σ.denote × τ.denote
  | sum σ τ => σ.denote ⊕ τ.denote
```

`Ty.denote` is a function from object-language types to Lean types: `nat` means `Nat`, an arrow means a Lean function type, `prod` means a Lean pair, and `sum` means Lean's sum type `⊕`. (`@[reducible]` lets Lean unfold it freely while checking types, so that a value of `nat.denote` can be used as a `Nat`.)

Variables are proofs of membership

A variable must carry its type. We use the indexed family that Chapter 6 introduced for contexts: `Var Γ τ` is a position in the list `Γ` at which the type `τ` occurs. `here` is the most recent binding, de Bruijn index 0, and `there` skips one; but unlike Chapter 7's bare numbers, a `Var` *proves* that the position holds the right type.

```
/-- `Var Γ τ` is a variable of type `τ` in the context `Γ`: a position in the
list `Γ` where `τ` occurs. `here` is the most recent binding (de Bruijn index
0); `there x` skips one binding. -/
inductive Var : List Ty → Ty → Type where
  | here {Γ : List Ty} {τ : Ty} : Var (τ :: Γ) τ
  | there {Γ : List Ty} {σ τ : Ty} : Var Γ τ → Var (σ :: Γ) τ
```

Terms are typing derivations

Now the terms. Each constructor is a typing rule: the indices in its result type are the conclusion, its arguments are the premises.

```
/-- `Term Γ τ` is a term of type `τ` in the context `Γ`. Each constructor is
a typing rule; an ill-typed term cannot even be written down. -/
inductive Term : List Ty → Ty → Type where
  | var {Γ τ} : Var Γ τ → Term Γ τ
  | lam {Γ σ τ} : Term (σ :: Γ) τ → Term Γ (σ ⇒ τ)
  | app {Γ σ τ} : Term Γ (σ ⇒ τ) → Term Γ σ → Term Γ τ
  | unit {Γ} : Term Γ .unit
  | tt {Γ} : Term Γ .bool
  | ff {Γ} : Term Γ .bool
  | ite {Γ τ} : Term Γ .bool → Term Γ τ → Term Γ τ → Term Γ τ
  | zero {Γ} : Term Γ .nat
  | succ {Γ} : Term Γ .nat → Term Γ .nat
  | iter {Γ τ} : Term Γ .nat → Term Γ τ → Term Γ (τ ⇒ τ) → Term Γ τ
  | pair {Γ σ τ} : Term Γ σ → Term Γ τ → Term Γ (.prod σ τ)
  | fst {Γ σ τ} : Term Γ (.prod σ τ) → Term Γ σ
  | snd {Γ σ τ} : Term Γ (.prod σ τ) → Term Γ τ
  | inl {Γ σ τ} : Term Γ σ → Term Γ (.sum σ τ)
  | inr {Γ σ τ} : Term Γ τ → Term Γ (.sum σ τ)
  | case {Γ σ τ ρ} :
      Term Γ (.sum σ τ) → Term (σ :: Γ) ρ → Term (τ :: Γ) ρ → Term Γ ρ
```

Compare `lam` with rule T-Lam: from a body of type `τ` in the context `σ :: Γ` (the context extended with the parameter), build a term of type `σ ⇒ τ` in `Γ`. `app` is T-App, `case` is T-Case, and so on. There is no annotation on `lam`: the parameter type `σ` is part of the term's type index. Because the constructors *are* the rules, a value of type `Term Γ τ` is at once a term and its typing derivation. Some example terms (`lit n` builds the numeral `n` from `zero` and `succ`):

```
/-- The numeral `n` as a term: `succ (succ (... zero))`. -/
def lit {Γ : List Ty} : Nat → Term Γ .nat
  | 0 => .zero
  | n + 1 => .succ (lit n)

/-- `twice = λf. λx. f (f x)`. In the body, `f` is `there here` (one binder
up) and `x` is `here`. -/
def twice : Term [] ((nat ⇒ nat) ⇒ nat ⇒ nat) :=
  .lam (.lam (.app (.var (.there .here)) (.app (.var (.there .here)) (.var .here))))

/-- `add = λm. λn. iter m n (λk. succ k)` -/
def add : Term [] (nat ⇒ nat ⇒ nat) :=
  .lam (.lam (.iter (.var (.there .here)) (.var .here) (.lam (.succ (.var .here)))))

/-- `swap = λp. (snd p, fst p)` -/
def swap {σ τ : Ty} : Term [] (.prod σ τ ⇒ .prod τ σ) :=
  .lam (.pair (.snd (.var .here)) (.fst (.var .here)))

/-- `toNat = λs. case s of inl n => n | inr b => if b then 1 else 0` -/
def toNat : Term [] (.sum nat bool ⇒ nat) :=
  .lam (.case (.var .here) (.var .here) (.ite (.var .here) (lit 1) (lit 0)))
```

An ill-typed term is not a term with a type error; it is a Lean type error. Applying the number zero to zero:

```
/--
error: Application type mismatch: The argument
  Term.zero
has type
  Term ?m.5 nat
but is expected to have type
  Term [] (nat ⇒ nat)
in the application
  Term.zero.app
-/
#guard_msgs in
example : Term [] .nat := .app .zero .zero
```

Lean's own type checker reports that `.zero` has type `Term _ nat`, where a function type `Term [] (nat ⇒ nat)` was needed. Self-application fails the same way: the variable `here` has type `σ` in the context `[σ]`, and the application needs it to have type `σ ⇒ τ`:

```
/--
error: Application type mismatch: The argument
  Var.here
has type
  Var (?m.11 :: ?m.10) ?m.11
but is expected to have type
  Var [σ] (σ ⇒ τ)
in the application
  Term.var Var.here
-/
#guard_msgs in
example (σ τ : Ty) : Term [] (σ ⇒ τ) := .lam (.app (.var .here) (.var .here))
```

An evaluator that cannot go wrong

An environment for `Γ` supplies a value of the right Lean type for each variable, and the evaluator maps a term of type `τ` to a Lean value of type `τ.denote`:

```
/-- An environment for `Γ`: a value of the right type for every variable. -/
abbrev Env (Γ : List Ty) : Type := ∀ {τ : Ty}, Var Γ τ → τ.denote
```

```
/-- The empty environment. There are no variables to look up. -/
def Env.nil : Env [] := fun x => nomatch x

/-- Extend an environment with a value for the new variable `here`. -/
def Env.cons {Γ : List Ty} {σ : Ty} (v : σ.denote) (ρ : Env Γ) : Env (σ :: Γ)
  | _, .here => v
  | _, .there x => ρ x
```

```
/-- `iterate f n a` applies `f` to `a`, `n` times. -/
def iterate {α : Type} (f : α → α) : Nat → α → α
  | 0, a => a
  | n + 1, a => f (iterate f n a)
```

```
/-- The evaluator: every term of type `τ` denotes a Lean value of type
`τ.denote`. There is no error case and no fuel: Lean checks that every case
is covered and that the recursion terminates. -/
def Term.eval {Γ : List Ty} {τ : Ty} : Term Γ τ → Env Γ → τ.denote
  | .var x, ρ => ρ x
  | .lam b, ρ => fun v => b.eval (ρ.cons v)
  | .app f a, ρ => f.eval ρ (a.eval ρ)
  | .unit, _ => ()
  | .tt, _ => true
  | .ff, _ => false
  | .ite c a b, ρ => if c.eval ρ then a.eval ρ else b.eval ρ
  | .zero, _ => 0
  | .succ n, ρ => n.eval ρ + 1
  | .iter n z s, ρ => iterate (s.eval ρ) (n.eval ρ) (z.eval ρ)
  | .pair a b, ρ => (a.eval ρ, b.eval ρ)
  | .fst p, ρ => (p.eval ρ).1
  | .snd p, ρ => (p.eval ρ).2
  | .inl a, ρ => .inl (a.eval ρ)
  | .inr b, ρ => .inr (b.eval ρ)
  | .case s l r, ρ =>
    match s.eval ρ with
    | .inl v => l.eval (ρ.cons v)
    | .inr v => r.eval (ρ.cons v)
```

Compare this with the Rust evaluator. There is no `Result`, no `Stuck`, no fuel, and no case for "applied something that is not a function". In the `app` case, `f.eval ρ` has type `(σ ⇒ τ).denote`, which *is* the Lean function type `σ.denote → τ.denote`, so applying it to `a.eval ρ` is just Lean function application. In the `ite` case, `c.eval ρ` is a `Bool`. The types guarantee that every case has what it needs, and Lean has checked that the match covers every constructor and that the recursion is structural. The evaluator is **total and safe by construction**: progress and preservation, which Chapter 9 must prove, are built into the types here. A `lam` evaluates to a genuine Lean function (`fun v => ...`), which we can apply to Lean functions directly:

```
/-- info: 10 -/
#guard_msgs in
#eval (Term.app (.app twice (.app add (lit 3))) (lit 4)).eval Env.nil

/-- The meaning of `twice` is a Lean function, which we can apply to Lean
functions. -/
example : twice.eval Env.nil (· * 3) 5 = 45 := rfl

/-- info: (true, 7) -/
#guard_msgs in
#eval (Term.app (swap (σ := nat) (τ := bool)) (.pair (lit 7) .tt)).eval Env.nil

/-- info: 1 -/
#guard_msgs in
#eval (Term.app toNat (.inr .tt)).eval Env.nil
```

This kind of evaluator, which maps the syntax of a language to mathematical objects (here, Lean values), is called a **denotational semantics**; Chapter 19 develops it into the interpretation of the STLC in a cartesian closed category.

β and η, proved

With the evaluator we can state β and η as theorems about meanings. β holds by definition, for functions and for the other types:

```
/-- β holds in the semantics, by computation: applying a λ means evaluating its
body with the argument's value bound to the parameter. -/
theorem eval_beta {Γ : List Ty} {σ τ : Ty} (b : Term (σ :: Γ) τ) (a : Term Γ σ)
    (ρ : Env Γ) : (Term.app (.lam b) a).eval ρ = b.eval (ρ.cons (a.eval ρ)) :=
  rfl

/-- So do the β-rules of the other types. -/
example {Γ : List Ty} {σ τ : Ty} (a : Term Γ σ) (b : Term Γ τ) (ρ : Env Γ) :
    (Term.fst (.pair a b)).eval ρ = a.eval ρ := rfl

example {Γ : List Ty} {τ : Ty} (z : Term Γ τ) (s : Term Γ (τ ⇒ τ)) (ρ : Env Γ) :
    (Term.iter .zero z s).eval ρ = z.eval ρ := rfl
```

`rfl` works because the left-hand side computes to the right-hand side by the equations of `eval`. η is more interesting, because it mentions the term `f.weaken`: the function `f`, moved into a context with one more variable (the parameter `x`) that it does not use. Weakening is an instance of **renaming**, which applies a type-preserving map on variables to every variable of a term:

```
/-- A renaming sends each variable of `Γ` to a variable of `Δ` with the same
type. -/
abbrev Ren (Γ Δ : List Ty) : Type := ∀ {τ : Ty}, Var Γ τ → Var Δ τ

/-- Going under a binder: the new variable stays `here`, the others move as
before, one position further out. -/
def Ren.ext {Γ Δ : List Ty} {σ : Ty} (r : Ren Γ Δ) : Ren (σ :: Γ) (σ :: Δ)
  | _, .here => .here
  | _, .there x => .there (r x)

/-- Apply a renaming to every variable of a term. -/
def Term.rename {Γ Δ : List Ty} (r : Ren Γ Δ) {τ : Ty} : Term Γ τ → Term Δ τ
  | .var x => .var (r x)
  | .lam b => .lam (b.rename r.ext)
  | .app f a => .app (f.rename r) (a.rename r)
  | .unit => .unit
  | .tt => .tt
  | .ff => .ff
  | .ite c a b => .ite (c.rename r) (a.rename r) (b.rename r)
  | .zero => .zero
  | .succ n => .succ (n.rename r)
  | .iter n z s => .iter (n.rename r) (z.rename r) (s.rename r)
  | .pair a b => .pair (a.rename r) (b.rename r)
  | .fst p => .fst (p.rename r)
  | .snd p => .snd (p.rename r)
  | .inl a => .inl (a.rename r)
  | .inr b => .inr (b.rename r)
  | .case e l rr => .case (e.rename r) (l.rename r.ext) (rr.rename r.ext)

/-- Weakening: a term of `Γ` is also a term of `σ :: Γ`. -/
def Term.weaken {Γ : List Ty} {σ τ : Ty} (t : Term Γ τ) : Term (σ :: Γ) τ :=
  t.rename Var.there
```

The key lemma says that evaluating a renamed term is evaluating the original term in an environment that "looks through" the renaming:

```
/-- Evaluating a renamed term in `ρ` is evaluating the original term in an
environment `ρ'` that agrees with `ρ` through the renaming. -/
theorem eval_rename {Γ Δ : List Ty} {τ : Ty} (t : Term Γ τ) (r : Ren Γ Δ)
    (ρ : Env Δ) (ρ' : Env Γ) (h : ∀ {σ : Ty} (x : Var Γ σ), ρ' x = ρ (r x)) :
    (t.rename r).eval ρ = t.eval ρ' := by
  -- Under a binder, the extended environments still agree.
  have ext {σ : Ty} (v : σ.denote) {Γ Δ : List Ty} (r : Ren Γ Δ) (ρ : Env Δ)
      (ρ' : Env Γ) (h : ∀ {σ : Ty} (x : Var Γ σ), ρ' x = ρ (r x)) :
      ∀ {σ' : Ty} (x : Var (σ :: Γ) σ'), (ρ'.cons v) x = (ρ.cons v) (r.ext x) := by
    intro σ' x
    cases x with
    | here => rfl
    | there x => exact h x
  induction t generalizing Δ with
  | var x => exact (h x).symm
  | lam b ih =>
    funext v
    exact ih r.ext (ρ.cons v) (ρ'.cons v) (ext v r ρ ρ' h)
  | case e l rr ihe ihl ihr =>
    simp only [Term.rename, Term.eval, ihe r ρ ρ' h]
    split
    · exact ihl r.ext _ _ (ext _ r ρ ρ' h)
    · exact ihr r.ext _ _ (ext _ r ρ ρ' h)
  -- The remaining cases: rename the subterms, then use the induction
  -- hypotheses (`ih r ρ ρ' h` rewrites a renamed subterm's value).
  | app f a ihf iha => simp only [Term.rename, Term.eval, ihf r ρ ρ' h, iha r ρ ρ' h]
  | ite c a b ihc iha ihb =>
    simp only [Term.rename, Term.eval, ihc r ρ ρ' h, iha r ρ ρ' h, ihb r ρ ρ' h]
  | succ n ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | iter n z s ihn ihz ihs =>
    simp only [Term.rename, Term.eval, ihn r ρ ρ' h, ihz r ρ ρ' h, ihs r ρ ρ' h]
  | pair a b iha ihb => simp only [Term.rename, Term.eval, iha r ρ ρ' h, ihb r ρ ρ' h]
  | fst p ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | snd p ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | inl a ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | inr b ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | unit | tt | ff | zero => rfl
```

The proof is by induction on the term. For a variable, the hypothesis `h` is exactly what is needed (`symm` flips the equation). For the binders, `lam` and `case`, the environments are extended by one value, and the local lemma `ext` shows, by cases on the variable, that the extended environments still agree through `r.ext`. For a `lam`, both sides are Lean functions, so `funext v` reduces the goal to their values at an arbitrary argument `v`. In the `case` case, `split` considers the two possible results of evaluating the scrutinee. The other cases unfold `rename` and `eval` (`simp only`) and rewrite each subterm with its induction hypothesis. η is now a short calculation:

```
/-- η holds in the semantics: `λx. f x` means the same as `f` (when `x` is
not free in `f`, which the weakening guarantees). -/
theorem eval_eta {Γ : List Ty} {σ τ : Ty} (f : Term Γ (σ ⇒ τ)) (ρ : Env Γ) :
    (Term.lam (.app f.weaken (.var .here))).eval ρ = f.eval ρ := by
  funext v
  -- After unfolding `eval`: (f.weaken).eval (ρ.cons v) v = f.eval ρ v.
  show (f.rename Var.there).eval (ρ.cons v) v = f.eval ρ v
  rw [eval_rename f Var.there (ρ.cons v) ρ (fun _ => rfl)]
```

After `funext v`, the goal is the value of both sides at `v`. `show` restates the left side as `eval` unfolds it: the renamed `f`, evaluated in the environment extended with `v`, applied to `v`. `eval_rename` with the renaming `Var.there` turns it into `f` evaluated in `ρ`, since looking up `there x` in `ρ.cons v` is looking up `x` in `ρ` (the `fun _ => rfl` proves that, for every variable, by computation).

Extrinsic versus intrinsic

The file `code/lean/Book/Ch08/Extrinsic.lean` builds the extrinsic version for comparison: raw terms, with the same constructors but no type indices, and a separate typing relation whose constructors are the same typing rules.

```
/-- The typing judgment `Γ ⊢ t : τ` as an inductive relation: one constructor
per typing rule. The λ rule does not say where `σ` comes from: any `σ` will
do, if the body can be typed with it (Curry style). -/
inductive HasType : List Ty → Raw → Ty → Prop where
  | var {Γ : List Ty} {k : Nat} {τ : Ty} :
      Γ[k]? = some τ → HasType Γ (.var k) τ
  | lam {Γ : List Ty} {t : Raw} {σ τ : Ty} :
      HasType (σ :: Γ) t τ → HasType Γ (.lam t) (σ ⇒ τ)
  | app {Γ : List Ty} {f a : Raw} {σ τ : Ty} :
      HasType Γ f (σ ⇒ τ) → HasType Γ a σ → HasType Γ (.app f a) τ
  | unit {Γ : List Ty} : HasType Γ .unit .unit
  | tt {Γ : List Ty} : HasType Γ .tt .bool
  | ff {Γ : List Ty} : HasType Γ .ff .bool
  | ite {Γ : List Ty} {c a b : Raw} {τ : Ty} :
      HasType Γ c .bool → HasType Γ a τ → HasType Γ b τ → HasType Γ (.ite c a b) τ
  | zero {Γ : List Ty} : HasType Γ .zero .nat
  | succ {Γ : List Ty} {n : Raw} : HasType Γ n .nat → HasType Γ (.succ n) .nat
  | iter {Γ : List Ty} {n z s : Raw} {τ : Ty} :
      HasType Γ n .nat → HasType Γ z τ → HasType Γ s (τ ⇒ τ) →
        HasType Γ (.iter n z s) τ
  | pair {Γ : List Ty} {a b : Raw} {σ τ : Ty} :
      HasType Γ a σ → HasType Γ b τ → HasType Γ (.pair a b) (.prod σ τ)
  | fst {Γ : List Ty} {p : Raw} {σ τ : Ty} :
      HasType Γ p (.prod σ τ) → HasType Γ (.fst p) σ
  | snd {Γ : List Ty} {p : Raw} {σ τ : Ty} :
      HasType Γ p (.prod σ τ) → HasType Γ (.snd p) τ
  | inl {Γ : List Ty} {a : Raw} {σ τ : Ty} :
      HasType Γ a σ → HasType Γ (.inl a) (.sum σ τ)
  | inr {Γ : List Ty} {b : Raw} {σ τ : Ty} :
      HasType Γ b τ → HasType Γ (.inr b) (.sum σ τ)
  | case {Γ : List Ty} {e l r : Raw} {σ τ ρ : Ty} :
      HasType Γ e (.sum σ τ) → HasType (σ :: Γ) l ρ → HasType (τ :: Γ) r ρ →
        HasType Γ (.case e l r) ρ
```

The two styles are connected by **erasure**: forgetting the types of an intrinsic term gives a raw term, and the theorem `erase_typed` says that the raw term is well typed, at the type the intrinsic term carried. The proof is an induction on the term in which every case is "apply the matching rule to the induction hypotheses", which makes the slogan precise: an intrinsic term *is* a typing derivation.

```
/-- A typed variable's index points at its type in the context. -/
theorem Var.index_lookup {Γ : List Ty} {τ : Ty} (x : Var Γ τ) : Γ[x.index]? = some τ := by
  induction x with
  | here => rfl
  | there x ih => simpa [Var.index] using ih

/-- Every intrinsically typed term, once its types are erased, is accepted by
the extrinsic typing relation, at the same type: the constructors of `Term`
*are* the typing rules. -/
theorem Term.erase_typed {Γ : List Ty} {τ : Ty} (t : Term Γ τ) :
    HasType Γ t.erase τ := by
  induction t with
  | var x => exact .var x.index_lookup
  | lam _ ih => exact .lam ih
  | app _ _ ihf iha => exact .app ihf iha
  | unit => exact .unit
  | tt => exact .tt
  | ff => exact .ff
  | ite _ _ _ ihc iha ihb => exact .ite ihc iha ihb
  | zero => exact .zero
  | succ _ ih => exact .succ ih
```

```
  | iter _ _ _ ihn ihz ihs => exact .iter ihn ihz ihs
  | pair _ _ iha ihb => exact .pair iha ihb
  | fst _ ih => exact .fst ih
  | snd _ ih => exact .snd ih
  | inl _ ih => exact .inl ih
  | inr _ ih => exact .inr ih
  | case _ _ _ ihe ihl ihr => exact .case ihe ihl ihr
```

The only case that needs a lemma is the variable: `Var.index_lookup` says that the de Bruijn index of a typed variable finds its type in the context. For `here` the index is 0 and the lookup returns the head of the list (`rfl`); for `there x` both the index and the context grow by one, and `simpa` (which simplifies the goal and the hypothesis and then closes one with the other) reduces the goal to the induction hypothesis.

```
/-- Erasing `twice` gives back the untyped `λ. λ. 1 (1 0)` of Chapter 7. -/
example : twice.erase = .lam (.lam (.app (.var 1) (.app (.var 1) (.var 0)))) := rfl

/-- So `HasType` accepts it, with the type that `twice` carries. -/
example : HasType [] twice.erase ((nat ⇒ nat) ⇒ nat ⇒ nat) := twice.erase_typed
```

Which style is better depends on what you want:

	Extrinsic (Chapters 3, 9)	**Intrinsic (this chapter)**
Ill-typed terms	exist; the checker rejects them	cannot be written
Typing	a separate relation `HasType`	the indices of `Term Γ τ`
Safety	a theorem to prove	true by construction
Evaluator	may get stuck (`Option`, `Stuck`)	total, no error cases
Good for	type checkers, studying errors	interpreters, semantics

Chapter 9 chooses the extrinsic style on purpose: to *prove* that well-typed programs do not go wrong, it needs to talk about programs that do, and to show that the typing rules keep evaluation away from them. The intrinsic style makes that theorem invisible, which is wonderful for building an interpreter and useless for studying why it works.

8.9 Putting it together: one function, three ways

Let us follow `twice (add 3) 4`, which should be 10, through all three implementations of this chapter.

In the `stlc` crate, the program is text. The parser builds a `Term`; the bidirectional checker synthesizes `Nat` for it, using C-Lam to type the unannotated `twice` against its annotation and S-App for each application; the evaluator builds closures and calls them:

```
twice (add 3) 4  :  Nat  ⇓  10
```

On the way, `add 3` became the closure `<closure λn. ... | m = 3>`, a body plus one captured value.

In Rust, `twice` is a generic function and `add` a closure returning a closure:

```
/// `twice = λf. λx. f (f x)`, for functions on `u64`.
fn twice(f: impl Fn(u64) -> u64) -> impl Fn(u64) -> u64 {
```

```
    move |x| f(f(x))
}
```

```
let add = |m: u64| move |n: u64| m + n;
let add3 = add(3);
println!("size of add(3) = {}", size_of_val(&add3));
println!("twice(add(3))(4) = {}", twice(add3)(4));
```

```
size of add(3) = 8
twice(add(3))(4) = 10
```

`add(3)` is a value of an anonymous closure type holding `m = 3`: 8 bytes, the same data as the crate's closure, laid out by the compiler instead of stored in a vector. `twice` is monomorphized for that type, so the two calls to `f` are direct.

In Lean, `twice` and `add` are intrinsically typed terms, and evaluation produces Lean values: the first `#eval` of the `run` block above printed 10. The types of the terms were checked by Lean's own type checker when we wrote them; the evaluator could not have failed.

Three implementations, one mathematical object: the term $\lambda f.\, \lambda x.\, f\,(f\,x)$ of type $(\texttt{Nat} \to \texttt{Nat}) \to \texttt{Nat} \to \texttt{Nat}$, typed by the derivation we wrote by hand at the start of the chapter.

8.10 Common confusions

"**A λ without an annotation has no type.**" It has many: $\lambda x.\, x$ has type $\tau \to \tau$ for every τ. What it lacks is a type that can be *synthesized* from the term alone. In checking mode, against an expected type, it is fine.

"**`move` makes a closure `FnOnce`.**" `move` only decides *how* variables are captured (by value). Which traits the closure implements depends on what its body does with the captures. A `move` closure that only reads is `Fn`.

"**`Fn` is the most general closure trait.**" It is the most *demanding* for the closure and the most *permissive* for the caller. A function that accepts `impl FnOnce` accepts every closure; one that accepts `impl Fn` accepts only those that can be called through a shared reference.

"**Closures are function pointers.**" A function pointer is a code address. A closure is a struct of captured values with a call method, and only a non-capturing closure can be coerced to a function pointer.

"**Two closures with the same code have the same type.**" Every closure expression has its own type. To mix them, use generics with one closure at a time, or a trait object.

"**Strong normalization means Rust programs terminate.**" It is a theorem about the STLC. Rust adds unrestricted recursion and `loop`, and its types promise nothing about termination.

"**Intrinsic typing makes the safety proof unnecessary.**" It moves the proof into the definition: the type of `Term.eval` *is* the safety statement, and Lean checked it when it accepted the definition. What you lose is the ability to talk about ill-typed terms at all.

8.11 Summary

- The STLC assigns types `Unit`, `Bool`, `Nat`, $\sigma \to \tau$, $\sigma \times \tau$ and $\sigma + \tau$ to λ-terms. Each type former has introduction and elimination rules; erased of terms, they are the rules of natural deduction.
- β-rules remove detours (an introduction followed by an elimination); η-rules say a value is determined by its uses. Clippy's `redundant_closure` is an η-reduction.
- Every well-typed STLC term terminates (strong normalization). $\lambda x.\, x\, x$ has no type, because $\sigma = \sigma \to \rho$ has no solution, a fact proved in Lean. Rust gives up termination to be Turing complete.
- Bidirectional checking splits typing into synthesis ($\Rightarrow$) and checking ($\Leftarrow$): introduction forms check, elimination forms synthesize, and a λ needs an expected type or an annotation. Rust's closures get their parameter types the same way.
- The `stlc` crate implements a bidirectional checker and an environment-based evaluator whose closures capture the free variables of their body, as Rust's do.
- Rust has several function types: zero-sized `fn` items, `fn` pointers, closures with unique anonymous types sized by their captures, generic `impl Fn`, and trait objects `dyn Fn`.
- Closures capture by shared reference, mutable reference or value; `move` forces by value. `Fn`, `FnMut` and `FnOnce` say how a closure may be called, and `FnOnce` is an affine function: usable at most once.
- Currying is free in the λ-calculus and costs allocations, clones and bounds in Rust.
- In Lean, intrinsically typed terms `Term Γ τ` are typing derivations; their denotational evaluator is total and safe by construction, and β and η hold as theorems. The extrinsic style, used in Chapter 9, keeps ill-typed terms around so that safety can be stated and proved.

8.12 Exercises

1. Write typing derivations for $\vdash \lambda p : \sigma \times \tau.\, (\mathtt{snd}\ p, \mathtt{fst}\ p) : \sigma \times \tau \to \tau \times \sigma$ and for $\vdash \lambda x : \sigma.\, \lambda y : \tau.\, x : \sigma \to \tau \to \sigma$. For each, say whether some variable is used twice, or not at all. *Hint:* in the second derivation y goes into the context and is never looked up; that is weakening (Chapter 6).
2. Extend the `stlc` crate with a primitive `isZero e`: give its rule in both modes, add it to the parser, checker and evaluator, and define `iszero` without `iter` in the example. Which mode should it be in? *Hint:* it is an elimination form for `Nat` with a known result type, so it synthesizes `Bool` after checking `e` against `Nat`.
3. Give a bidirectional rule for `case` in checking mode and one in synthesis mode, and explain why the synthesis rule must *check* the second branch against the type synthesized from the first. Find both in `check.rs`. *Hint:* compare with rule T-If in Chapter 1, and with how Rust reports the type of a `match` whose arms disagree.
4. Write a Rust function `compose3` that composes three closures, first with `impl Fn` arguments and result, then with `Box<dyn Fn>`. Use `size_of_val` to compare the sizes of the results, and explain the difference. *Hint:* the `impl Fn` result is a closure holding three closures; the boxed version is a fat pointer, whatever it holds.
5. Which of `Fn`, `FnMut`, `FnOnce` does each closure implement? `|v: &mut Vec<u32>| v.push(1)`, `move || drop(buffer)`, `|| counter.get()` (with `counter: Cell<u32>`), `move |x| x + offset`. Check

your answers by passing each to functions that require each trait. *Hint:* the parameter `v` is not a capture; `Cell::get` takes `&self`.

6. In Lean, add a constructor `isZero : Term Γ .nat → Term Γ .bool` to `Intrinsic.lean` (in a copy), extend `eval`, `rename` and the proof of `eval_rename`, and prove `(Term.app (.lam (.isZero (.var .here))) (lit 0)).eval Env.nil = true` by `rfl`. *Hint:* `eval` maps it to `n.eval ρ == 0`; the `eval_rename` case is like `succ`.